DEVOPS INTERNSHIP

Task 3: Infrastructure as Code (IaC)

Provision a Local Docker Container Using Terraform

Technology	Purpose
Terraform	Define and provision infrastructure as code
Docker	Run the local container
Nginx	Web server used to verify deployment

Prepared as a practical implementation guide with Terraform code, execution steps, verification, cleanup, and interview preparation.

1. Objective and Scope

The task objective is to provision a local Docker container using Terraform. The task sheet asks for a Docker provider, Terraform initialization and application, destruction of the infrastructure, a plan before applying, and inspection of Terraform state.

Resources created: Terraform manages an Nginx Docker image resource and a Docker container named `terraform-nginx-demo`. Host port 8080 is mapped to container port 80.

Project architecture

```
Local machine
  Terraform CLI
      |
      v
  Docker provider
      |
      v
  Docker Engine ---- pulls ----> nginx:alpine
      |
      v
  Container: terraform-nginx-demo
      |
      +---- host localhost:8080 -> container port 80
```

2. Terraform Configuration — main.tf

Create a file named `main.tf` with the following content:

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}

provider "docker" {
  # Uses the local Docker Engine by default.
}

resource "docker_image" "nginx" {
  name         = "nginx:alpine"
  keep_locally = true
}

resource "docker_container" "web" {
  name  = "terraform-nginx-demo"
  image = docker_image.nginx.image_id

  ports {
    internal = 80
    external = 8080
  }

  restart = "unless-stopped"
}
```

The container references the image resource, so Terraform can infer that the image must be available before the container is created.

3. Prerequisites

• Install Terraform CLI (version 1.5 or newer).

• Install Docker Engine or Docker Desktop and make sure the Docker daemon is running.

• Internet access is needed to download the Docker provider and, on the first run, the Nginx image.

• Open a terminal in the directory containing main.tf.

```
docker version
terraform version
```

The Docker version command should display both client and server information. If Docker cannot connect to its daemon, start Docker before proceeding.

4. Execution Procedure

Step 1 — Format and initialize

```
terraform fmt
terraform init
```

Expected result: Terraform formats the configuration and initializes the working directory, downloading the required Docker provider.

Evidence: Save a screenshot as `screenshots/01-terraform-init.png`.

Step 2 — Validate and plan

```
terraform validate
terraform plan -out=tfplan
```

Validation checks the configuration. The plan previews the changes without applying them. Review the proposed resources before continuing.

Evidence: Save a screenshot as `screenshots/02-terraform-plan.png`.

Step 3 — Apply

```
terraform apply tfplan
```

Terraform executes the saved plan. On success, the image and container resources should be managed in Terraform state.

Evidence: Save a screenshot as `screenshots/03-terraform-apply.png`.

Step 4 — Verify the web server and container

```
docker ps --filter name=terraform-nginx-demo
curl -I http://localhost:8080
```

Open `http://localhost:8080` in a browser. The default Nginx welcome page should appear. The curl request will normally return `HTTP/1.1 200 OK`.

Step 5 — Inspect Terraform state

```
terraform state list
terraform state show docker_container.web
terraform show
```

The state list should include `docker_image.nginx` and `docker_container.web` after a successful apply.

Evidence: Save a screenshot as `screenshots/04-container-and-state.png`.

5. Destroy and Clean Up

```
terraform destroy
```

Review the proposed deletion and type `yes` when prompted. The container will be removed from Docker. Since `keep_locally = true`, the Nginx image remains in the local image cache.

```
docker ps -a --filter name=terraform-nginx-demo
terraform state list
```

After successful destruction, the test container should no longer exist and the state list for this configuration should be empty.

Evidence: Save a screenshot as `screenshots/05-terraform-destroy.png`.

6. Execution Log Template

Record the real output from your own machine; do not use invented or assumed output as proof.

Initialization

Command: `terraform fmt && terraform init`

Actual output / notes: _____

_____

Validation and plan

Command: `terraform validate && terraform plan -out=tfplan`

Actual output / notes: _____

_____

Apply

Command: `terraform apply tfplan`

Actual output / notes: _____

_____

Verification

Command: `docker ps --filter name=terraform-nginx-demo && curl -I http://localhost:8080 && terraform state list`

Actual output / notes: _____

_____

Destroy

Command: `terraform destroy`

Actual output / notes: _____

_____

7. Troubleshooting

Docker daemon unavailable: Start Docker Desktop or the Docker service, then rerun docker version.

Permission denied on Docker socket (Linux): Check that Docker is running. sudo may be used for diagnosis. Membership in the docker group grants root-level-equivalent privileges, so only grant it to trusted users.

Host port 8080 is already in use: Change external = 8080 to another free port such as 8081, then run terraform plan and terraform apply.

Provider or image download fails: Check internet connectivity and DNS, then retry terraform init or terraform apply.

Container name already exists: Inspect docker ps -a. Remove an old test container only if you have verified it is safe to remove.

8. Interview Questions and Answers

1. What is IaC?

Infrastructure as Code means defining and managing infrastructure using machine-readable configuration files instead of manual setup. It improves repeatability, reviewability, and automation.

2. How does Terraform work?

Terraform reads configuration, initializes providers, compares desired configuration with state and real resources, creates a plan, and applies the required changes through providers.

3. What is a Terraform state file?

terraform.tfstate records Terraform's mapping between configuration resources and real resources, including attributes and metadata. It helps Terraform decide what to create, update, or destroy.

4. Difference between plan and apply?

terraform plan previews proposed changes. terraform apply executes the changes, with approval when required or by using a saved plan.

5. What are providers?

Providers are plugins that let Terraform communicate with platforms such as Docker, Azure, AWS, or Kubernetes.

6. What is resource dependency?

A dependency defines the order in which resources must be managed. Terraform can infer dependencies when one resource references another, as the container references the image here.

7. How do you handle secret variables?

Use environment variables, sensitive input variables, or a secrets manager; restrict access to state; and never commit secrets. The sensitive flag hides values in normal CLI output but does not necessarily remove them from state.

8. Benefits of Terraform?

Repeatability, automation, version control, previewable changes, dependency management, reusable modules, and multi-provider support.

9. GitHub Submission

Create a new repository, for example `terraform-docker-iac-task`. Add main.tf, README.md, execution-log-template.md, .gitignore, and your genuine screenshots. Do not commit state files, the .terraform directory, or plan files.

```
git init
git add main.tf README.md execution-log-template.md .gitignore screenshots/
git commit -m "Complete Terraform Docker IaC task"
git branch -M main
git remote add origin YOUR_GITHUB_REPOSITORY_URL
git push -u origin main
```

Replace YOUR_GITHUB_REPOSITORY_URL with the URL of the repository you created, then submit that URL using the internship submission form.

10. Completion Checklist

■ Terraform and Docker versions checked.

■ terraform init completed.

■ terraform validate and terraform plan reviewed.

■ terraform apply completed.

■ Container verified through Docker and localhost:8080.

■ Terraform state inspected.

■ terraform destroy completed.

■ Actual terminal logs and screenshots added.

■ Project pushed to a new GitHub repository.

■ No state files, plan files, credentials, or .terraform directory committed.